

NAPQES: A Single-Primitive Authenticated Encryption Scheme Built from HMAC-SHA256

Abdeslam Jakjoud¹ and Karim Amor¹

¹QAegis, France

July 2026

Abstract

We present NAPQES, an Authenticated Encryption with Associated Data (AEAD) scheme whose security reduces exclusively to the pseudorandomness of HMAC-SHA256. Unlike AES-GCM and ChaCha20-Poly1305, NAPQES employs no finite-field arithmetic, group operations, or AES S-box permutations, and is therefore not subject to Shor’s algorithm or hidden-subgroup quantum attacks. The construction keys two independently sampled secrets: an arithmetic-layer key $\mathbf{k}$, an ordered tuple of primes driving a prime-indexed token cipher, and a uniform 256-bit secret $\mathbf{sk}$ that keys every HMAC domain derivation used for noise injection, plaintext padding, keystream masking, and message authentication. The nonce is derived synthetically as $N = \text{HMAC}(\mathbf{sk}, 0\mathbf{x}0\mathbf{A} \parallel \text{be8}(|A|) \parallel A \parallel M)$, in the manner of AES-GCM-SIV, rather than sampled independently at random. Because $\mathbf{k}$ and $\mathbf{sk}$ are sampled independently and only $\mathbf{sk}$ ever keys an HMAC call, we prove IND-CPA-det (the restricted-query notion appropriate to a deterministic, misuse-resistant scheme’s synthetic nonce), INT-CTXT, and IND-CCA-det security (Theorems 4.7, 4.10, 4.11) under the standard, uniform-key PRF assumption on HMAC-SHA256 — with no key-entropy term and no non-standard key-distribution assumption — and show that nonce misuse (re-encrypting an identical associated-data/message pair) degrades gracefully to disclosed message-equality leakage rather than key recovery. Ciphertext length is proved to be a deterministic function of the padded-length bucket alone (Lemma 3.8), never of message content, nonce, or associated data. We discuss the post-quantum security level, provide performance measurements and implementation notes across Python, Rust, and C reference implementations, and report a 30-vector Known-Answer Test corpus together with a NIST SP 800-22 Rev. 1a statistical analysis. A reference implementation, KAT corpus, and fuzz harnesses are published at <https://github.com/Synergeon-epineon/NAPQES>.

Contents

1	Introduction	3
2	Related Work	3
3	Construction	4
3.1	Notation	4
3.2	Key Generation	4
3.3	Domain-Separated HMAC Derivation	5
3.4	Noise Probability	6
3.5	Plaintext Padding	6
3.6	Token Construction	6
3.7	Synthetic Nonce	7

3.8	Wire Format	8
3.9	The NAPQES Algorithm Triple	8
4	Security Analysis	9
4.1	PRF Advantage and Truncation	9
4.2	IND-CPA-det Security	10
4.3	INT-CTXT: Ciphertext Integrity	12
4.4	IND-CCA-det Security	12
5	Post-Quantum Analysis	13
6	Comparison with AES-GCM, ChaCha20-Poly1305, and Ascon	13
7	Implementation	14
7.1	Python Reference Implementation	14
7.2	Rust Implementation	14
7.3	C Implementation	14
7.4	Constant-Time Considerations	14
8	NIST SP 800-22 Statistical Analysis	14
9	Known-Answer Tests	15
10	Fuzz Testing	15
11	Known Caveats	15
12	Conclusion	16

1 Introduction

Modern AEAD standards—AES-GCM [1] and ChaCha20-Poly1305 [2]—have achieved widespread deployment. Their security, however, ultimately depends on algebraic structures (finite-field multiplication for GCM, add-rotate-XOR chains for ChaCha20) whose resilience against quantum adversaries running variants of Shor’s algorithm is an active research question [7].

NAPQES takes a more conservative approach: the *only* cryptographic primitive in the construction is HMAC-SHA256, approved under NIST FIPS 198-1 and FIPS 180-4. The security of the entire scheme therefore reduces to the pseudorandomness of HMAC-SHA256—a well-studied assumption with no known quantum speedup beyond Grover’s $O(\sqrt{N})$ search, which reduces the effective security level from 256 bits to approximately 128 bits while remaining far above all recommended thresholds.

Contributions.

1. We fully specify NAPQES (Section 3): an independent HMAC-domain-derivation secret $\mathbf{sk}$ and arithmetic-layer key $\mathbf{k}$, sampled independently, together with a synthetic, message-bound nonce $N = \text{HMAC}(\mathbf{sk}, 0\mathbf{x}0\mathbf{A} \parallel \text{be8}(|A|) \parallel A \parallel M)$ (Definition 3.7) and a fixed-width, per-bucket-ceiling wire format (Section 3.8) under which ciphertext length depends only on the padded-length bucket.
2. We give a formal AEAD algorithm triple $(\text{KeyGen}, \text{Enc}, \text{Dec})$ (Definition 3.9) and prove correctness (Definition 3.10).
3. We prove IND-CPA-det (Definition 4.4), INT-CTXT (Theorem 4.10), and IND-CCA-det (Theorem 4.11) security (Theorem 4.7) under the *standard, uniform-key* PRF assumption on HMAC-SHA256, with no key-entropy term: because $\mathbf{sk}$ is sampled uniformly and independently of $\mathbf{k}$, every reduction below may sample $\mathbf{k}$ on its own to run the arithmetic layer while forwarding every HMAC call to an external PRF oracle keyed by $\mathbf{sk}$, with no simulation gap of any kind.
4. We prove ciphertext length is a deterministic function of the padded-length bucket alone (Lemma 3.8) — never of message content within a bucket, the nonce, or the associated data.
5. We provide a post-quantum analysis (Section 5) and a comparison with AES-GCM, ChaCha20-Poly1305, and Ascon (Section 6).
6. We report NIST SP 800-22 Rev. 1a statistical test results on ciphertext output (Section 8), a 30-vector KAT corpus (Section 9), and a fuzz harness covering the full decoder pipeline (Section 10).
7. We discuss known caveats (Section 11): the 16-bit plaintext length cap, the fixed ciphertext expansion factor, the padding length-bucket leak, and the scope of what each of $\mathbf{sk}$ and $\mathbf{k}$ actually protects.

2 Related Work

AES-GCM [1]. AES-GCM combines AES-CTR for confidentiality with a GHASH polynomial MAC over $\text{GF}(2^{128})$. The Forbidden Attack [8] and nonce-reuse attacks [9] demonstrate that the algebraic structure of GHASH can be exploited when the polynomial MAC key is recovered (e.g., under nonce misuse), algebraically recovering the authentication key. NAPQES uses HMAC-SHA256 for authentication, which does not admit the Forbidden Attack.

ChaCha20-Poly1305 [2]. ChaCha20 is an ARX cipher whose security is not reduced to a well-studied hardness assumption; Poly1305 is again a polynomial MAC, subject to the same class of Forbidden Attacks as GHASH when the one-time key is reused. Neither ChaCha20 nor Poly1305 is approved under current FIPS standards.

Ascon [3]. Ascon (NIST LWC winner) is a sponge-based AEAD targeting lightweight environments. Its security is based on the hardness of distinguishing the Ascon permutation from a random permutation, which involves xor-and-rotate operations over 64-bit words. Ascon is not yet in a FIPS-approved family.

HMAC-based constructions. HKDF [10] and HMAC-DRBG [11] demonstrate that HMAC-SHA256 is a sufficient basis for key derivation and pseudorandom generation. NAPQES extends this philosophy to the full AEAD context.

Deterministic and misuse-resistant AE. Rogaway and Shrimpton [13] formalise deterministic authenticated encryption and the corresponding restricted-query security notion; AES-GCM-SIV follows the same synthetic-IV philosophy NAPQES adopts for its nonce.

3 Construction

3.1 Notation

Throughout this paper:

- $\parallel$ denotes byte concatenation.
- $\text{HMAC}(K, M)$ denotes HMAC-SHA256 keyed with K over message M .
- $\text{be4}(x)$, $\text{be5}(x)$, $\text{be8}(x)$ denote the 4-, 5-, and 8-byte big-endian encodings of the non-negative integer x (each field is fixed-width; x is assumed to fit).
- $[0:n]_{\text{bek}}$ denotes the first n bytes of a 32-byte HMAC-SHA256 digest, interpreted as an unsigned big-endian integer in $[0, 2^k)$ where $k = 8n$.
- $\mathcal{P}$ denotes the set of prime integers in $[10^6, 9.9 \times 10^6]$, a fixed public system parameter with $|\mathcal{P}| \approx 586,000$.

3.2 Key Generation

A NAPQES key pair is $(\mathbf{k}, \text{sk})$, sampled by $\text{KeyGen}(1^\lambda)$ as follows.

Definition 3.1 (Key generation). *Draw $\mathbf{k} = (k_1, \dots, k_K)$, an ordered tuple of K distinct primes sampled uniformly, without replacement, from $\mathcal{P}$ via a cryptographically secure DRBG (default $K = 10$); independently, draw $\text{sk} \xleftarrow{\$} \{0, 1\}^{256}$ uniformly via a CSPRNG. No function relates sk to $\mathbf{k}$. The serialised arithmetic key is*

$$\text{key_bytes}(\mathbf{k}) = \text{be5}(k_1) \parallel \text{be5}(k_2) \parallel \dots \parallel \text{be5}(k_K).$$

Remark 3.2 (Concrete sampling algorithm). $\mathcal{P}$ is fixed as a public, canonical ascending enumeration $\mathcal{P} = \{p_0 < p_1 < \dots < p_{|\mathcal{P}|-1}\}$ (e.g. a precomputed sieve table), so “the idx -th prime of $\mathcal{P}$ ” is well-defined and identical across implementations. Index sampling is by rejection: $\text{SAMPLEINDEX}()$ draws a uniform 32-bit unsigned integer x from the CSPRNG, sets $L = \lfloor 2^{32}/|\mathcal{P}| \rfloor \cdot |\mathcal{P}|$, and rejects and redraws x whenever $x \geq L$; once $x < L$, it returns $\text{idx} = x \bmod |\mathcal{P}|$, which is then exactly uniform over $\{0, \dots, |\mathcal{P}| - 1\}$ with no modulo bias. k_1 is set to $p_{\text{SAMPLEINDEX}()};$ for $j = 2, \dots, K$, k_j is drawn by repeatedly invoking $\text{SAMPLEINDEX}()$ and

rejecting (redrawing) any result equal to an already-chosen $k_1, \dots, k_{j-1}$, until a fresh value is obtained. This realises exactly the uniform distribution over ordered tuples of K distinct elements of $\mathcal{P}$ assumed by the $H_\infty(\mathbf{k})$ computation below.

The min-entropy of $\mathbf{k}$ as sampled above is

$$H_\infty(\mathbf{k}) = \log_2 \left(\frac{|\mathcal{P}|!}{(|\mathcal{P}| - K)!} \right) \approx K \log_2 |\mathcal{P}| \approx 19.16 K \text{ bits},$$

so $H_\infty(\mathbf{k}) \approx 191.6$ bits at the default $K = 10$ (key-space size $\approx 4.8 \times 10^{57} \approx 2^{191.6}$).

Remark 3.3 (The role of $\mathbf{k}$ and $\mathbf{sk}$ are not symmetric). $\mathbf{sk}$ is the only secret that ever keys an HMAC call in this construction (Section 3.3); $\mathbf{k}$ is used exclusively, directly, and un-keyed by the arithmetic token map $c \mapsto c \cdot k_j + a$ (Section 3.6). Consequently, none of the security theorems of Section 4 depend in any way on $H_\infty(\mathbf{k})$: the PRF assumption below is stated, and used, against the standard uniform-key distribution of $\mathbf{sk}$ only. K therefore carries no security-theorem-driven minimum in this construction (contrast a single-secret design in which the arithmetic key also keys HMAC, which would require a minimum K to keep an offline key-guessing term negligible). $K = 10$ is retained as the default purely for the traffic-analysis-resistance contribution of the multi-prime token construction (Lemma 3.8 and the discussion in Section 11) and as a matter of general defense-in-depth, not because any theorem below requires it. Section 11 discusses, conversely, what protecting $\mathbf{k}$ actually buys once $\mathbf{sk}$ is known.

3.3 Domain-Separated HMAC Derivation

All internal values are computed as

$$\text{Derive}_d(\mathbf{sk}, N, \text{ctx}) = \text{HMAC}(\mathbf{sk}, d \| N \| \text{ctx}),$$

where d is a 1-byte domain separator and every variable-width field inside ctx is length-prefixed (e.g. $\text{be8}(|A|) \| A$ for AAD A). Domain $0x0A$ is the single exception: it does not take a nonce as input, since it is the derivation that *produces* the nonce (Definition 3.7); its input shape is instead $0x0A \| \text{be8}(|A|) \| A \| M$.

Domain	Function	Context	Output
0x00	Noise position	$\text{be5}(ct_pos)$	Boolean
0x01	Real-token addend	$\text{be5}(real_idx)$	$[1, k_i - 1]$
0x02	Noise probability	(none)	$[0.75, 0.99]$
0x03	Auth tag	$\text{be8}(aad) \ aad \ \tilde{B}$	32 bytes
0x04	Noise character	$\text{be5}(ct_pos)$	$[32, 127]$
0x05	Noise-token addend	$\text{be5}(ct_pos)$	$[1, k_i - 1]$
0x06	Padding codepoint	$\text{be4}(pad_idx)$	$[32, 126]$
0x07	Keystream block	$\text{be4}(block_idx)$	32-byte block
0x0A	Synthetic nonce	$\text{be8}(A) \ A \ M$	16 bytes

Table 1: Domain-separated HMAC derivation schedule. Every domain is keyed by $\mathbf{sk}$ only. The nonce N is applied uniformly by Derive_d for $d \neq 0x0A$ and is therefore omitted from the Context column above.

Lemma 3.4 (Domain separation). *For every pair of distinct queries to $\text{Derive}_\bullet(\mathbf{sk}, \cdot, \cdot)$ made anywhere in this construction, the two underlying HMAC-SHA256 inputs are unconditionally distinct.*

Proof. Two inputs with different domain bytes $d_1 \neq d_2$ differ at byte position 0, regardless of any other field, so cross-domain distinctness is unconditional. Within a fixed domain $d \neq 0x0A$:

bytes 1..16 carry the nonce N , so for fixed N , distinctness reduces to injectivity of $\text{ctx} \mapsto \text{ctx}$; for domains $\{0x00, 0x01, 0x02, 0x04, 0x05, 0x06, 0x07\}$ this is a fixed-width positional counter (or empty), trivially injective; for domain $0x03$, $\text{ctx} = \text{be8}(|A|) \| A \| \tilde{B}$, and because $\text{be8}(|A|)$ fixes $|A|$, the split between A and $\tilde{B}$ is unambiguous, so distinct $(A, \tilde{B})$ pairs never collide. Within domain $0x0A$, whose input is $0x0A \| \text{be8}(|A|) \| A \| M$ with no nonce field at all, the same length-prefix argument shows $\text{be8}(|A|) \| A \| M$ is injective in (A, M) . Hence every semantically distinct derivation, in every domain, queries $\text{HMAC}(\text{sk}, \cdot)$ at a distinct input. $\square$

3.4 Noise Probability

The noise probability is realised entirely with fixed-width integer arithmetic; no floating-point representation is used anywhere in this construction. Fix the public integer constants

$$\theta_{\min} = \lfloor 0.75 \times 2^{64} \rfloor, \quad \theta_{\max} = \lfloor 0.99 \times 2^{64} \rfloor,$$

and let $\tau = \text{Derive}_{0x02}(\text{sk}, N, \epsilon)[0 : 8]_{\text{be64}} \in [0, 2^{64})$ be the raw 64-bit unsigned integer HMAC output (no division involved). The per-message noise threshold is the 64-bit integer

$$\theta(N) = \theta_{\min} + \left\lfloor \frac{\tau \cdot (\theta_{\max} - \theta_{\min})}{2^{64}} \right\rfloor,$$

computed with a single 128-bit intermediate product and one integer division, so that $\theta(N)$ is bit-identical across all conforming implementations regardless of platform or language. Every informal reference elsewhere in this paper to a “noise probability $p \in [0.75, 0.99]$ ” (e.g. Table 1) denotes this integer threshold, i.e. $p = \theta(N)/2^{64}$.

3.5 Plaintext Padding

Let $n = |P|$ (plaintext length in Unicode codepoints). Define

$$B = \max(16, 2^{\lceil \log_2(n+1) \rceil}), \quad P_d = [n \gg 8, n \wedge 0xFF] \| P \| \mathbf{pad},$$

where $\mathbf{pad} = (\mathbf{pad}_0, \dots, \mathbf{pad}_{B-n-1})$ is a $(B - n)$ -codepoint sequence with

$$\mathbf{pad}_i = (\text{Derive}_{0x06}(\text{sk}, N, \text{be4}(i))[0:4]_{\text{be32}} \bmod 95) + 32 \in [32, 126].$$

The total padded length is $R = n + 2 + (B - n) = B + 2$ codepoints; we call B the *padding bucket* of the message and R the *real-token count*. Because B depends only on n (never on the codepoint values themselves), two messages of equal codepoint length always share the same padding bucket and the same real-token count.

3.6 Token Construction

For each padded codepoint c , define the following primitives, each an explicit reduction of a 32-byte Derive_d output into its target range:

Definition 3.5 (Token-construction primitives). *For a key element k , position counters ct_pos , $real_idx$:*

$$\begin{aligned} is_noise(ct_pos) &= \mathbf{1}[\text{Derive}_{0x00}(\text{sk}, N, \text{be5}(ct_pos))[0:8]_{\text{be64}} < \theta(N)], \\ noise_char(ct_pos) &= (\text{Derive}_{0x04}(\text{sk}, N, \text{be5}(ct_pos))[0:4]_{\text{be32}} \bmod 96) + 32, \\ noise_addend(ct_pos, k) &= (\text{Derive}_{0x05}(\text{sk}, N, \text{be5}(ct_pos))[0:4]_{\text{be32}} \bmod (k - 1)) + 1, \\ real_addend(real_idx, k) &= (\text{Derive}_{0x01}(\text{sk}, N, \text{be5}(real_idx))[0:4]_{\text{be32}} \bmod (k - 1)) + 1. \end{aligned}$$

Remark 3.6 (Modulo bias). *Each finite-range reduction above (and the padding-codepoint reduction of Section 3.5) draws a value pseudo-uniformly from $[0, 2^{32})$ and reduces it modulo a small integer m . For any $m \leq 2^{32}$, every residue’s probability deviates from $1/m$ by at most a factor $m/2^{32}$: below 2^{-25} for the padding codepoint ($m = 95$) and noise character ($m = 96$), and below 2^{-8} for the addends ($m = k - 1 < 2^{24}$). This bias is statistically dominated by the underlying PRF advantage against HMAC-SHA256 and is negligible for the security claims of Section 4; no rejection sampling is required. The noise threshold $\theta(N)$ (Section 3.4) is exempt from this discussion entirely: it is computed from fixed public integer constants via exact integer arithmetic rather than a modulo reduction, so no residue bias applies to it at all, and the prime-index sampling of Remark 3.2 is bias-free by rejection rather than by statistical dominance.*

Real and noise tokens are integers $c \cdot k + a$ for codepoint c (or a noise character), key element $k = \mathbf{k}[i \bmod K]$, and addend a from Definition 3.5. Because k is prime and every addend lies in $[1, k - 1]$, $\gcd(a, k) = 1$ and hence $\gcd(\text{token}, k) = 1$ for every token, real or noise.

To keep ciphertext length independent of the message-derived nonce’s particular noise realisation (Section 3.8), the number of *consecutive* noise tokens emitted before each real token is capped at a fixed constant $\text{MAX_NOISE_RUN} = 19$, and the total token count is padded, with additional filler tokens structurally identical to genuine noise tokens, up to a fixed ceiling of exactly $R \cdot (\text{MAX_NOISE_RUN} + 1)$ tokens per message:

Listing 1: Token emission (encoding)

```

1  for c in padded:                                # R codepoints total
2      noise_run = 0
3      while noise_run < MAX_NOISE_RUN and is_noise(ct_pos):
4          k = key[real_idx % K]
5          emit noise_char(ct_pos) * k + noise_addend(ct_pos, k)
6          ct_pos += 1; noise_run += 1
7      k = key[real_idx % K]
8      emit c * k + real_addend(real_idx, k)
9      ct_pos += 1; real_idx += 1
10
11 # Ceiling padding: append filler tokens (identical construction to
12 # genuine noise tokens) until exactly R * (MAX_NOISE_RUN + 1) tokens
13 # have been emitted in total.
14 ceiling = R * (MAX_NOISE_RUN + 1)
15 while len(tokens) < ceiling:
16     k = key[real_idx % K]
17     emit noise_char(ct_pos) * k + noise_addend(ct_pos, k)
18     ct_pos += 1

```

Because the ceiling depends only on R (hence only on the padding bucket B), the total token count $N_{\text{tok}} = R \cdot (\text{MAX_NOISE_RUN} + 1)$ is, by construction, a deterministic function of the padding bucket alone — regardless of sk , N , A , or the specific message (Lemma 3.8).

3.7 Synthetic Nonce

Definition 3.7 (Synthetic nonce). *For $\text{sk} \in \{0, 1\}^{256}$, $A \in \{0, 1\}^*$, $M \in \mathcal{M}$ (encoded as UTF-8 bytes),*

$$N(\text{sk}, A, M) = \text{HMAC}(\text{sk}, 0x0A \parallel \text{be8}(|A|) \parallel A \parallel M)[0:16].$$

This is a synthetic IV in the manner of RFC 5297 SIV / AES-GCM-SIV: it is a deterministic function of the key and the input, not sampled independently at random, so re-encrypting an identical (A, M) pair always reproduces the identical nonce (and hence the identical ciphertext)

— the standard, disclosed message-equality trade-off of deterministic, misuse-resistant encryption (Definition 4.4).

3.8 Wire Format

Each token is serialised in a fixed-width 8-byte field,

$$B = \text{be8}(\text{token}[0]) \parallel \text{be8}(\text{token}[1]) \parallel \cdots \parallel \text{be8}(\text{token}[N_{\text{tok}} - 1]),$$

so $|B| = 8N_{\text{tok}}$ bytes, with $N_{\text{tok}} = R \cdot (\text{MAX_NOISE_RUN} + 1)$ as fixed in Section 3.6. B is XOR-masked with a keystream derived from domain $0x07$:

$$\tilde{B} = B \oplus \text{ks}(N)[0 : |B|], \quad \text{ks}(N) = \text{Derive}_{0x07}(\text{sk}, N, \text{be4}(0)) \parallel \text{Derive}_{0x07}(\text{sk}, N, \text{be4}(1)) \parallel \cdots,$$

generating only as many 32-byte blocks as needed to cover $|B|$ bytes (truncating the final block). The authentication tag is

$$T = \text{Derive}_{0x03}(\text{sk}, N, \text{be8}(|A|) \parallel A \parallel \tilde{B}),$$

a 32-byte HMAC-SHA256 value binding the (possibly empty) associated data A . The ciphertext is

$$C = N \parallel \tilde{B} \parallel T,$$

a 16-byte nonce, the masked token blob, and the 32-byte tag. A decryptor rejects any input with $|C| < 48$ bytes outright (parse-time floor); the true minimum length of a well-formed ciphertext is larger, since every message — including the empty string — pads to at least $B = 16$ codepoints ($R = 18$ real tokens), giving $N_{\text{tok}} = 18 \cdot 20 = 360$, $|B| = 2880$ bytes, and $|C| = 16 + 2880 + 32 = 2928$ bytes at minimum.

Lemma 3.8 (Ciphertext length is a deterministic function of the padding bucket). *For any $\mathbf{k}$, sk , A , M ,*

$$|C| = 16 + 8R(\text{MAX_NOISE_RUN} + 1) + 32, \quad R = B(|M|) + 2,$$

where $B(\cdot)$ is the padding-bucket function of Section 3.5. In particular $|C|$ depends on M only through $B(|M|)$ — never through sk , N , A , or the specific codepoints of M .

Proof. Immediate from the construction of Section 3.6: N_{tok} is fixed, by the ceiling-padding step, to exactly $R \cdot (\text{MAX_NOISE_RUN} + 1)$ tokens regardless of the natural noise realisation, and $R = B(|M|) + 2$ depends only on the codepoint *count* of M (Section 3.5). The wire format then adds a constant 16-byte nonce and 32-byte tag, and masking (XOR) does not change $|B|$. $\square$

3.9 The NAPQES Algorithm Triple

Definition 3.9 (NAPQES AEAD algorithm triple). *NAPQES is the triple $(\text{KeyGen}, \text{Enc}, \text{Dec})$ over the following typed spaces:*

- *Key space $\mathcal{K} \times \{0, 1\}^{256}$: $\mathcal{K}$ is the set of ordered K -tuples of distinct elements of $\mathcal{P}$ (Section 3.2).*
- *AAD space $\mathcal{A} = \{0, 1\}^*$: the AAD length is encoded as $\text{be8}(|A|)$ throughout (Section 3.3), so any $|A| < 2^{64}$ bytes is supported unambiguously, with no practical ceiling.*
- *Message space $\mathcal{M}$: finite sequences of Unicode codepoints of length at most $\text{MAX_PLAINTEXT_CODEPOINTS} - 1$, the limit imposed by the 2-codepoint big-endian length prefix written during padding (Section 3.5).*
- *Ciphertext space $\mathcal{C} = \{0, 1\}^*$: every output of Enc has length $16 + 8R(\text{MAX_NOISE_RUN} + 1) + 32$ bytes for $R = B(|M|) + 2$ (Lemma 3.8); the parse-time rejection floor is 48 bytes, but the true minimum length of a well-formed ciphertext is 2928 bytes.*

$\text{KeyGen}(1^\lambda) \rightarrow (\mathbf{k}, \text{sk})$. As in Definition 3.1.

$\text{Enc}(\mathbf{k}, \text{sk}, A, M) \rightarrow C$. (1) compute $N = N(\text{sk}, A, M)$ (Definition 3.7); (2) pad M to P_d (Section 3.5); (3) run the token-emission loop of Section 3.6 over P_d using (sk, N) to obtain the token vector $(\text{token}[0], \dots, \text{token}[N_{\text{tok}}-1])$; (4) serialise B ; (5) mask $\tilde{B} = B \oplus \text{ks}(N)$; (6) compute $T = \text{Derive}_{0x03}(\text{sk}, N, \text{be8}(|A|) \| A \| \tilde{B})$; (7) output $C = N \| \tilde{B} \| T$. The public encryption oracle is $\text{Encrypt}(\mathbf{k}, \text{sk}, A, M) \triangleq \text{Enc}(\mathbf{k}, \text{sk}, A, M)$ — a pure function of (A, M) , since N is computed synthetically rather than sampled.

$\text{Dec}(\mathbf{k}, \text{sk}, A, C) \rightarrow M$ or $\perp$. (1) if $|C| < 48$, output $\perp$; (2) parse $C = N \| \tilde{B} \| T'$ with N the leading 16 bytes and T' the trailing 32 bytes; (3) let $N_{\text{tok}} = (|C| - 48)/8$ and, if this is not a non-negative integer or not divisible by $\text{MAX_NOISE_RUN} + 1$, output $\perp$; otherwise set $R = N_{\text{tok}}/(\text{MAX_NOISE_RUN} + 1)$; (4) compute $T = \text{Derive}_{0x03}(\text{sk}, N, \text{be4}(|A|) \| A \| \tilde{B})$; (5) if $T \neq T'$ (compared in constant time), output $\perp$; (6) else unmask $B = \tilde{B} \oplus \text{ks}(N)$, deserialise the fixed-width tokens, and invert the token-emission loop using the same (sk, N) -derived noise-position oracle and addends, stopping once R real tokens have been extracted (discarding any trailing filler); strip the length prefix and padding; output M .

Definition 3.10 (Correctness). For every λ , every $(\mathbf{k}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$, every $A \in \mathcal{A}$, $M \in \mathcal{M}$:

$$\text{Dec}(\mathbf{k}, \text{sk}, A, \text{Enc}(\mathbf{k}, \text{sk}, A, M)) = M.$$

Proof. Enc 's token-emission loop and Dec 's loop-inversion step both compute the noise-position oracle, addends, and padding codepoints as pure, deterministic functions of $(\text{sk}, N, \text{position})$; given the same sk and the (deterministically re-derivable, once M is recovered token-by-token) N , both algorithms compute identical intermediate values at every position, so token emission and inversion are exact inverses whenever $T = T'$. Correctness is additionally checked empirically by the cross-language KAT corpus and the full regression suite across the Python, Rust, and C implementations (Section 7). $\square$

Remark 3.11 (Authentication-failure output $\perp$). The formal $\perp$ output of Dec is realised in all three reference implementations as a raised exception (`ValueError` in Python/Rust, an equivalent error code in C) rather than a sentinel return value — a standard, semantically equivalent realisation of $\perp$ (cf. most software AEAD APIs, e.g. `cryptography.hazmat`).

4 Security Analysis

Every theorem in this section is unconditional under the standard, uniform-key PRF assumption on HMAC-SHA256 stated below: because sk is sampled uniformly and independently of $\mathbf{k}$, every reduction constructed in this section may sample $\mathbf{k}$ on its own (to run the arithmetic layer) while forwarding every HMAC call to an external oracle keyed by the challenger's sk — the reduction never needs to extract sk from $\mathbf{k}$, or vice versa, so no simulation gap and no non-standard key-distribution assumption arises. NAPQES does not define a standalone MAC verification key separate from sk , so EUF-CMA unforgeability of the tag and INT-CTXT unforgeability of the ciphertext coincide here.

4.1 PRF Advantage and Truncation

Definition 4.1 (HMAC-SHA256 PRF advantage). Let $F = \text{HMAC-SHA256} : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^{256}$. For a PPT distinguisher $\mathcal{B}$ with oracle access to either $F(\kappa, \cdot)$ for $\kappa \xleftarrow{\$} \{0, 1\}^{256}$, or a function $\rho : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$ sampled uniformly from all functions of that type,

$$\text{Adv}_F^{\text{PRF}}(\mathcal{B}) = \left| \Pr[\mathcal{B}^{F(\kappa, \cdot)} = 1] - \Pr[\mathcal{B}^{\rho(\cdot)} = 1] \right|.$$

Assumption 4.2 (Uniform-key HMAC-SHA256 PRF). $\text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B})$ is negligible in the security parameter for every PPT $\mathcal{B}$, for κ drawn uniformly from $\{0,1\}^{256}$ (matching how sk is sampled, Definition 3.1).

Lemma 4.3 (Truncation preserves the PRF property). For any fixed truncation length $\tau \leq 256$, writing $\text{trunc}_\tau(x) = x[0:\tau]$, $\text{Adv}_{\text{trunc}_\tau \circ F}^{\text{PRF}}(\mathcal{B}) = \text{Adv}_F^{\text{PRF}}(\mathcal{B})$.

Proof. ρ is a uniformly random function, so $\text{trunc}_\tau \circ \rho$ is itself uniformly random over $\{0,1\}^{8\tau}$, independently across distinct inputs. Any distinguisher against $\text{trunc}_\tau \circ F$ yields, by truncating every oracle answer it receives before continuing to run unmodified, a distinguisher against F with identical advantage. $\square$

This covers every truncated derivation used by this construction (nonce $[0:16]$, addend/character $[0:4]$, noise threshold $[0:8]$) under Assumption 4.2 alone.

4.2 IND-CPA-det Security

Because $\text{Encrypt}(\mathbf{k}, \text{sk}, A, M)$ is a pure function of (A, M) (Section 3.9), the standard, unrestricted left-or-right IND-CPA game is not achievable by any deterministic scheme: an adversary who queries the encryption oracle on one of its own two challenge messages trivially learns the challenge bit by comparing the result to the challenge ciphertext. This is not a defect of the construction — it is the standard, well-understood property of deterministic, misuse-resistant encryption (Rogaway and Shrimpton [13]; cf. AES-GCM-SIV) — and the correct notion of confidentiality for such a scheme is the restricted-query game below.

Definition 4.4 (IND-CPA-det security). For a PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, define $\text{Exp}_{\mathcal{A}}^{\text{ind-cpa-det}}(\lambda)$:

1. $(\mathbf{k}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$.
2. $\mathcal{A}_1$ is given adaptive oracle access to $\text{Encrypt}(\mathbf{k}, \text{sk}, \cdot, \cdot)$ and, after at most q total queries (counted jointly with step 4), outputs $(A^*, m_0, m_1, \text{st})$ with $|m_0| = |m_1|$ (codepoint count).
3. The challenger samples $b \xleftarrow{\$} \{0,1\}$ and returns $c^* \leftarrow \text{Encrypt}(\mathbf{k}, \text{sk}, A^*, m_b)$.
4. $\mathcal{A}_2(\text{st}, c^*)$, with continued oracle access, outputs a bit b' .
5. The experiment outputs 1 iff $b' = b$.

$\mathcal{A}$ must additionally respect two restrictions, standard for deterministic/misuse-resistant AE [13], or the experiment aborts with $\mathcal{A}$ losing: (i) no repeated queries — $\mathcal{A}$ never queries the encryption oracle twice on an identical (A, M) pair, in either phase; (ii) no challenge-message queries — writing (A^*, m_0, m_1) for $\mathcal{A}_1$'s output, $\mathcal{A}$ never queries the encryption oracle, in either phase, on (A^*, m_0) or (A^*, m_1) . The advantage of $\mathcal{A}$ is $\text{Adv}^{\text{IND-CPA-det}}(\mathcal{A}) = |\Pr[\text{Exp}_{\mathcal{A}}^{\text{ind-cpa-det}}(\lambda) = 1] - \frac{1}{2}|$.

Lemma 4.5 (Equal padded length is derived). $|m_0| = |m_1|$ forces m_0, m_1 to share the same padding bucket B and the same real-token count R (Section 3.5), with certainty — this is a direct consequence of B depending only on codepoint count.

Lemma 4.6 (Nonce collision). For any two distinct queried pairs $(A_1, M_1) \neq (A_2, M_2)$ under the same sk , there is a PRF adversary $\mathcal{B}$ with similar running time such that $\Pr[N(\text{sk}, A_1, M_1) = N(\text{sk}, A_2, M_2)] \leq \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}) + 2^{-128}$.

Proof. By Lemma 3.4, $(A_1, M_1) \neq (A_2, M_2)$ implies the two domain $0x0A$ inputs are distinct, so replacing $\text{HMAC}(\text{sk}, \cdot)$ by a truly random function ρ (a PRF hop of advantage $\text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B})$) makes the two 128-bit truncated outputs independent and uniform, colliding with probability exactly 2^{-128} . $\square$

Theorem 4.7 (IND-CPA-det). For any PPT adversary $\mathcal{A}$ respecting Definition 4.4's restrictions and making at most q encryption queries, there exists a PRF adversary $\mathcal{B}_1$ with similar running

time such that

$$\text{Adv}^{\text{IND-CPA-det}}(\mathcal{A}) \leq \text{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1) + \frac{q^2}{2^{128}},$$

against the standard, uniform-key HMAC-SHA256 PRF assumption, with no key-guessing or key-entropy term.

Proof. We proceed via a sequence of two hybrid games.

Game G_0 (real). The challenger samples $(\mathbf{k}, \text{sk})$ and answers queries via the real Enc , which calls $F(\text{sk}, \cdot)$ for all internal derivations.

Game G_1 (PRF replacement). Identical, except $F(\text{sk}, \cdot)$ is replaced by a truly random function $\rho : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$, sampled lazily.

Lemma 4.8. *There is a PRF adversary $\mathcal{B}_1$, running in time $\mathcal{O}(T_{\mathcal{A}})$, such that $|\Pr[\mathcal{A} \text{ wins } G_0] - \Pr[\mathcal{A} \text{ wins } G_1]| \leq \text{Adv}_F^{\text{PRF}}(\mathcal{B}_1)$.*

Proof. $\mathcal{B}_1$ is given an oracle $\mathcal{O}$ that is either $F(\text{sk}, \cdot)$ or $\rho(\cdot)$. Because sk is sampled independently of $\mathbf{k}$ (Definition 3.1), $\mathcal{B}_1$ samples its own $\mathbf{k}'$ via the prime-sampling step of KeyGen to run the arithmetic layer of Enc directly, and forwards every domain-derivation call (including domain $0\text{x}0\text{A}$) to $\mathcal{O}$. $\mathcal{B}_1$ never needs to know sk (or the challenger's hidden key) to run this simulation: when $\mathcal{O} = F(\text{sk}, \cdot)$ the simulation is distributed identically to G_0 ; when $\mathcal{O} = \rho$, identically to G_1 . $\square$

Nonce collision under G_1 . Let Coll be the event that the challenge nonce N^* collides with some queried nonce $N_1, \dots, N_q$. By Definition 4.4's restriction (ii), every queried pair (A_i, M_i) is distinct from the challenge pair (A^*, m_b) regardless of b , so Lemma 4.6 applies validly to every pair $\{(A_i, M_i), (A^*, m_b)\}$; a union bound over the q queries (together with pairwise collisions among $N_1, \dots, N_q$) gives $\Pr[\text{Coll}] \leq q^2/2^{128}$ (the $\text{Adv}^{\text{PRF}}(\mathcal{B}_1)$ term from Lemma 4.6 is already accounted for by $\mathcal{B}_1$ above).

Lemma 4.9 (Hiding). *In G_1 , conditioned on $\overline{\text{Coll}}$, $\Pr[\mathcal{A} \text{ wins } G_1 \mid \overline{\text{Coll}}] = \frac{1}{2}$.*

Proof. Under $\overline{\text{Coll}}$, $N^* \notin \{N_1, \dots, N_q\}$. Every challenge-phase derivation (domains $0\text{x}00$ – $0\text{x}07$) uses an input prefixed by its domain byte and then N^* ; by Lemma 3.4 none of these inputs collides with any query-phase input (different nonce at the fixed offset) or with any other domain's input (different domain byte). Since ρ is truly random, all challenge-phase outputs are therefore fresh, uniform, and independent of the adversary's view. In particular the keystream $\text{ks}(N^*)$ (domain $0\text{x}07$) is independent of the token blob B it masks (which is itself assembled entirely from domains $0\text{x}00, 0\text{x}01, 0\text{x}02, 0\text{x}04, 0\text{x}05, 0\text{x}06$), so the one-time-pad identity applies: for fixed B , $\tilde{B} = B \oplus \text{ks}(N^*)$ is uniform over $\{0, 1\}^{8|B|}$. By Lemma 4.5, m_0, m_1 share the same padding bucket and real-token count R ; by Lemma 3.8, $N_{\text{tok}} = R(\text{MAX_NOISE_RUN} + 1)$ is therefore identical for m_0 and m_1 (indeed for *any* nonce), so $|B|$ — and hence the distribution of $\tilde{B}$ — is identical regardless of b . The tag $T = \rho(0\text{x}03 \| N^* \| \dots)$ is a fresh uniform 32-byte value, also independent of b . Hence $c^* = (N^*, \tilde{B}, T)$ is jointly uniform over its domain, independently of b , giving the adversary no information about b . $\square$

Combining: $\Pr[\mathcal{A} \text{ wins } G_1] \leq \Pr[\text{Coll}] + \frac{1}{2} \leq q^2/2^{128} + \frac{1}{2}$, so

$$\text{Adv}^{\text{IND-CPA-det}}(\mathcal{A}) = |\Pr[\mathcal{A} \text{ wins } G_0] - \frac{1}{2}| \leq \text{Adv}_F^{\text{PRF}}(\mathcal{B}_1) + \frac{q^2}{2^{128}}. \quad \square$$

4.3 INT-CTXT: Ciphertext Integrity

Theorem 4.10 (INT-CTXT). *For any PPT adversary $\mathcal{A}$ making at most q encryption queries and at most $q_v \geq 1$ adaptive decryption/verification queries, there exists a PRF adversary $\mathcal{B}_2$ with similar running time such that*

$$\text{Adv}^{\text{INT-CTXT}}(\mathcal{A}) \leq \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_2) + \frac{q_v}{2^{256}}.$$

Freshness is measured over the pair (c, aad) against $\{(c_1, \text{aad}_1), \dots, (c_q, \text{aad}_q)\}$, the encryption oracle’s own outputs; there is no q -dependent (encryption-query-count) term, since Case 2 below is resolved deterministically rather than probabilistically.

Proof. Write a forgery attempt as $c^* = (N^*, \tilde{B}^*, T^*)$ with tag input $x^* = 0x03 \| N^* \| \text{be4}(|\text{aad}^*|) \| \text{aad}^* \| \tilde{B}^*$.

Case 1: N^* was not used in any encryption query. By Lemma 3.4, x^* is distinct from every query-phase domain $0x03$ input (different nonce at the fixed offset) and from every other domain’s input, so x^* was never submitted to $\text{HMAC}(\text{sk}, \cdot)$. Construct $\mathcal{B}_2$, given an oracle f (real or random), exactly as $\mathcal{B}_1$ in Lemma 4.8: sample $\mathbf{k}'$ independently, forward every HMAC call to f , and output “real” iff some forgery attempt’s candidate tag matches $f(x^*)$. In the ideal world $f(x^*)$ is uniform and independent of $\mathcal{A}$ ’s view, so each of the q_v attempts succeeds by blind guessing with probability exactly 2^{-256} ; a union bound gives $\Pr[\text{forge in Case 1}] \leq \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_2) + q_v/2^{256}$.

Case 2: $N^* = N_i$ for some query i . If $x^* \neq x_i$ for every such i (which, by injectivity of the length-prefixed AAD encoding, covers both $\tilde{B}^* \neq \tilde{B}_i$ and the $\tilde{B}^* = \tilde{B}_i \wedge \text{aad}^* \neq \text{aad}_i$ replay-with-new-AAD sub-case), x^* was never queried and Case 1’s argument applies verbatim. If instead $x^* = x_i$ for some i , injectivity forces $\text{aad}^* = \text{aad}_i \wedge \tilde{B}^* = \tilde{B}_i$: if also $T^* = T_i$ then $(c^*, \text{aad}^*) = (c_i, \text{aad}_i)$, excluded by freshness; if $T^* \neq T_i$, the recomputed tag equals $T_i \neq T^*$ and Dec rejects unconditionally. Either sub-case contributes no successful, freshness-respecting forgery, and both branches are resolved deterministically via Lemma 3.4’s injectivity — N^* is chosen adaptively by $\mathcal{A}$ after observing $N_1, \dots, N_q$, so no probabilistic collision term applies to Case 2.

Combining both cases yields the stated bound. $\square$

4.4 IND-CCA-det Security

Theorem 4.11 (IND-CCA-det). *For any PPT adversary making at most q encryption queries and q_d decryption queries, and respecting Definition 4.4’s restrictions (i)–(ii) on its own encryption-oracle queries, there exist PRF adversaries $\mathcal{B}_1, \mathcal{B}_2$ with similar running time such that*

$$\text{Adv}^{\text{IND-CCA-det}}(\mathcal{A}) \leq \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_1) + \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_2) + \frac{q^2}{2^{128}} + \frac{q_d}{2^{256}}.$$

Proof sketch. We apply the Bellare–Namprempre composition theorem [12]: a scheme that is both IND-CPA-det (Theorem 4.7) and INT-CTXT (Theorem 4.10) is IND-CCA-det. Concretely, define H_0 as the real IND-CCA-det game (encryption and adaptive decryption oracles, with only the exact pair (c^*, A^*) forbidden from the decryption oracle) and H_1 as H_0 with the decryption oracle replaced by a table lookup that answers a query (c, aad) with the recorded plaintext if (c, aad) was previously output by the encryption oracle (including the challenge triple, once issued) and $\perp$ otherwise. H_0 and H_1 agree on every query except a fresh, valid forgery — event D — so $|\Pr[H_0] - \Pr[H_1]| \leq \Pr[D]$. An INT-CTXT forger constructed by forwarding $\mathcal{A}$ ’s q encryption queries and its own $(q+1)$ -th (used to produce the challenge

c^*), and every decryption query in both phases, to a real INT-CTXT challenger simulates H_0 perfectly, so $\Pr[D] \leq \text{Adv}^{\text{PRF}}(\mathcal{B}_2) + q_d/2^{256}$ by Theorem 4.10 (no encryption-query-count term). In H_1 , the table-backed decryption oracle discloses nothing $\mathcal{A}$ could not reconstruct itself from its own queries (the one exception, the challenge triple, is never looked up since (c^*, A^*) is forbidden), so H_1 reduces directly to the IND-CPA-det game of Theorem 4.7, giving $|\Pr[H_1] - \frac{1}{2}| \leq \text{Adv}^{\text{PRF}}(\mathcal{B}_1) + q^2/2^{128}$. Combining the two bounds yields the stated inequality. $\square$

5 Post-Quantum Analysis

SHA-256 and Grover’s algorithm. Grover’s algorithm [4] provides a quadratic speedup for unstructured search, requiring $\approx 2^{n/2}$ oracle queries against an n -bit target. Applied to SHA-256 preimage search, this reduces the nominal security level from 256 to approximately 128 bits. This 2^{128} figure is a bound on *sequential query depth*, not a parallelizable work-factor: the Bennett–Bernstein–Brassard–Vazirani (BBBV) optimality result [5] shows no quantum algorithm can search an unstructured space of size N in fewer than $\Omega(\sqrt{N})$ oracle queries, and Zalka [6] shows that distributing Grover’s search across S machines yields only a $\sqrt{S}$ speedup — the depth stays $\approx 2^{n/2}/\sqrt{S}$ and the total work is unchanged, unlike classical brute force, which parallelizes linearly. At 2^{128} sequential oracle queries, this remains far beyond feasibility and above the 112-bit minimum NIST recommends for post-2030 systems.

No Shor-applicable structure. NAPQES does not use discrete logarithms, integer factorisation, or any algebraic group structure. Shor’s algorithm and related quantum algorithms provide no speedup against the construction.

Two separate key-space figures. Because sk is the sole HMAC key and $\mathbf{k}$ is used only by the un-keyed arithmetic layer (Remark 3.3), the two secrets have different, non-interchangeable security roles and are best reported separately rather than as one conflated figure:

- sk : uniform 256 bits, classical exhaustive search 2^{256} , post-Grover sequential depth $\approx 2^{128}$ — this is the figure relevant to Assumption 4.2 and to every theorem of Section 4.
- $\mathbf{k}$: $\approx 2^{191.6}$ ordered-tuple key-space size at the default $K = 10$ (post-Grover $\approx 2^{95.8}$), relevant only to the structural/traffic-analysis contribution of the multi-prime token construction — *not* to any theorem in Section 4, none of which depend on $H_\infty(\mathbf{k})$ (Remark 3.3).

Both figures are non-parallelizable sequential-depth bounds by the same BBBV [5]/Zalka [6] argument. Section 11 discusses the practical consequence of this asymmetry: protecting sk is necessary and sufficient for confidentiality and integrity; protecting $\mathbf{k}$ in addition provides no independent confidentiality margin once sk is compromised.

6 Comparison with AES-GCM, ChaCha20-Poly1305, and Ascon

The primary trade-off is ciphertext expansion. The token-emission loop caps consecutive noise tokens at $\text{MAX_NOISE_RUN} = 19$ (Section 3.6) and pads every message to exactly $R \cdot (\text{MAX_NOISE_RUN} + 1) = 20R$ tokens, so ciphertext expansion is a hard, deterministic $20\times$ per real codepoint — never a range, and never dependent on the per-message noise realisation (Lemma 3.8). Compared to no expansion for AES-GCM and ChaCha20-Poly1305, this overhead is acceptable in applications where the traffic pattern does not rely on ciphertext length confidentiality beyond the padding bucket (Section 11).

Unlike AES-GCM and ChaCha20-Poly1305, where nonce reuse degrades confidentiality but does not expose the key, NAPQES’s synthetic, message-bound nonce (Definition 3.7) makes accidental reuse across *distinct* (A, M) pairs a negligible HMAC-SHA256-collision event (Lemma 4.6)

Property	NAPQES	AES-GCM	ChaCha
Primitive basis	HMAC-SHA256	AES + GHASH	ChaCha
FIPS-approved	Yes	Yes	
Shor-applicable structure	No	No	
Forbidden Attack risk	No	Yes (nonce reuse)	Yes (I)
Nonce-reuse key recovery	No (Definition 3.7, disclosed message-equality only)	Yes (GCM key)	Yes (I)
Ciphertext expansion	20× (fixed)	1×	
Key size	≥ 82 bytes (k+sk)	32 bytes	

Table 2: Comparison of NAPQES with established AEAD schemes.

rather than a birthday-bound or DRBG-failure event; re-querying with an *identical* (A, M) pair reproduces the identical ciphertext deterministically, disclosing only plaintext equality — the standard, disclosed MRAE trade-off (cf. AES-GCM-SIV), not a confidentiality or key-recovery break.

7 Implementation

7.1 Python Reference Implementation

The Python reference implementation (`napqes.py`) is pure Python 3.10+, with no dependencies beyond the standard library (`hmac`, `hashlib`, `secrets`, `base64`). Key generation and the AEAD entry points are `generate_v8_key`, `encrypt_bytes_v8`, and `decrypt_bytes_v8`. All KAT vectors are derived from this implementation.

7.2 Rust Implementation

A Rust port (`rust/src/lib.rs`) mirrors the Python implementation (`generate_v8_key`, `encrypt_bytes_v8`, `decrypt_bytes_v8`). Unit tests cover round-trip correctness, tamper/AAD-mismatch rejection, same-message determinism (the disclosed MRAE trade-off), distinct-message nonce distinctness, and ciphertext-length determinism across varied AAD and keys (Lemma 3.8).

7.3 C Implementation

A C port (`C/napqes.c`) mirrors the same construction (`napqes_encrypt_bytes_v8`, `napqes_decrypt_bytes_v8`).

7.4 Constant-Time Considerations

Authentication tag comparison uses `hmac.compare_digest` (Python) and `constant_time_eq` from the `subtle` crate (Rust), providing constant-time equality on the tag bytes.

8 NIST SP 800-22 Statistical Analysis

We applied the full NIST SP 800-22 Rev. 1a Statistical Test Suite [14] to 10^7 bits of NAPQES ciphertext output generated by encrypting a cycling corpus of printable-ASCII plaintext with a 10-element prime key. All 15 eligible SP 800-22 tests passed at the $\alpha = 0.01$ significance level. Results are reproducible using `tests/sts_pipeline.py` in the reference implementation repository.

9 Known-Answer Tests

The reference KAT corpus (`tests/kat/v6_vectors.json`) contains 30 vectors covering the empty message, single characters, boundary block sizes, 1-element and 10-element keys, non-empty and binary AAD, multiple nonces for the same message, same-message determinism, distinct-message nonce distinctness, authentication-tag tamper, undersized ciphertext, wrong key, and wrong AAD. Conforming implementations must produce byte-identical ciphertext for every positive vector and raise a matching error for every negative vector. The generator (`tests/gen_kats.py`) can be used to verify parity on any port.

10 Fuzz Testing

Two fuzz harnesses are provided. The Python harness (`tests/fuzz_atheris.py`, using `atheris`) targets `decrypt_bytes_v8` and the fixed-width token decoder, verifying that no exception other than `ValueError` escapes the public API. The Rust harness (`rust/fuzz/fuzz_targets/decode_bytes.rs`, using `cargo-fuzz`) targets the Rust `decrypt_bytes_v8` entry point with multiple key sizes and AAD values. No panics were observed across 10^6 corpus entries in either harness.

11 Known Caveats

Plaintext length cap. The block padding scheme stores the plaintext length as a 2-byte big-endian integer, capping messages at 65,535 codepoints. Exceeding the cap raises an error immediately; there is no silent truncation. Callers requiring larger messages must split at the application layer.

Ciphertext expansion. Fixed at exactly $20\times$ per real codepoint (Section 6), a deliberate traffic-analysis-resistance design trade-off, not a bug.

Padding length-bucket leakage. Ciphertext length reveals the power-of-two padding bucket $\{16, 32, \dots, 65536\}$, leaking up to $\lceil \log_2 n \rceil$ bits of length information (Lemma 3.8); it does not reveal the exact codepoint count within a bucket, and — because the token ceiling is fixed per bucket — it does not vary with `sk`, the nonce, or the associated data, so no averaging attack across repeated encryptions of one message under varying AAD can extract more than the bucket itself. Callers needing full length-hiding must still layer a fixed-frame transport.

Scope of `sk` and `k` compromise. `sk` is the sole cryptographic secret in this construction: every HMAC domain derivation — the noise schedule, the padding, the per-position addends, the keystream, and the tag — is keyed by `sk` alone (Section 3.3). Consequently, an adversary who compromises `sk` but not `k` can recover any real token’s plaintext codepoint directly: the per-position addend’s raw HMAC output depends only on `sk`, so for a given token the adversary can, for each of the $\approx 586,000$ candidate primes in $\mathcal{P}$, compute the corresponding addend and check whether the resulting quotient is an exact integer in a valid codepoint range — an $\mathcal{O}(|\mathcal{P}|)$ trial-division search that succeeds in microseconds. `k`’s secrecy therefore adds **no** independent confidentiality margin once `sk` is known: compromise of `sk` alone is equivalent to a full break, regardless of whether `k` also leaked. Operationally, `k` and `sk` must therefore be protected with equal rigor, as a single unit — not because splitting them across separate storage or trust domains buys compartmentalized defense-in-depth (it does not), but because `sk` compromise alone is already fatal. This is consistent with, and does not weaken, the security theorems of Section 4, which assume the entire key material is secret from the adversary, exactly as for any symmetric AEAD scheme (a leaked AES-256 key is likewise a full break of AES-GCM).

12 Conclusion

We have presented NAPQES, an AEAD scheme whose entire security reduces to the pseudorandomness of HMAC-SHA256. The construction avoids all algebraic structures targeted by known quantum algorithms, relies solely on FIPS-approved primitives, and keys two independently sampled secrets — an arithmetic-layer prime tuple $\mathbf{k}$ and a uniform 256-bit HMAC key $\mathbf{sk}$ — together with a synthetic, message-bound nonce that makes accidental nonce reuse across distinct messages negligible and degrades identical-message reuse only to disclosed plaintext equality. We prove IND-CPA-det, INT-CTXT, and IND-CCA-det security under the standard, uniform-key PRF assumption on HMAC-SHA256, with no key-entropy term and no simulation gap, and prove that ciphertext length is a deterministic function of the padded-length bucket alone. The scheme has been validated against a 30-vector Known-Answer Test corpus, the full NIST SP 800-22 statistical suite, and a fuzz harness covering the complete decoder pipeline across Python, Rust, and C reference implementations.

Remaining open limitations are documented in Section 11: the fixed $20\times$ ciphertext expansion inherent in the noise-injection design, the 16-bit plaintext length cap, the padding length-bucket leak, and the scope of what $\mathbf{sk}$ and $\mathbf{k}$ compromise each imply. Future work includes a TVLA side-channel analysis of the constant-time Rust implementation, a FIPS 140-3 module boundary definition, and a performance evaluation on embedded ARM Cortex-M4 and RISC-V targets.

References

- [1] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D, November 2007.
- [2] Y. Nir, A. Langley, *ChaCha20 and Poly1305 for IETF Protocols*, RFC 7539, May 2015.
- [3] C. Dobraunig, M. Eichlseder, F. Mendel, M. Schl  ffer, *Ascon v1.2: Lightweight Authenticated Encryption and Hashing*, Journal of Cryptology, 34(3), 2021.
- [4] L. K. Grover, *A fast quantum mechanical algorithm for database search*, Proceedings of STOC '96, pp. 212–219, 1996.
- [5] C. H. Bennett, E. Bernstein, G. Brassard, U. Vazirani, *Strengths and Weaknesses of Quantum Computing*, SIAM Journal on Computing, 26(5), pp. 1510–1523, 1997.
- [6] C. Zalka, *Grover’s quantum searching algorithm is optimal*, Physical Review A, 60(4), pp. 2746–2751, 1999.
- [7] NIST, *Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process*, NIST IR 8413, July 2022.
- [8] A. Joux, *Authentication failures in NIST version of GCM*, NIST Comment, 2006.
- [9] T. Iwata, K. Ohashi, K. Minematsu, *Breaking and Repairing GCM Security Proofs*, CRYPTO 2012, LNCS 7417, pp. 31–49.
- [10] H. Krawczyk, P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869, May 2010.
- [11] E. Barker, J. Kelsey, *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*, NIST SP 800-90A Rev 1, June 2015.
- [12] M. Bellare, C. Namprempre, *Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm*, ASIACRYPT 2000, LNCS 1976, pp. 531–545.

- [13] P. Rogaway, T. Shrimpton, *A Provable-Security Treatment of the Key-Wrap Problem*, EUROCRYPT 2006, LNCS 4004, pp. 373–390.
- [14] A. Rukhin, J. Soto, J. Nechvatal, et al., *A Statistical Test Suite for Random and Pseudo-random Number Generators for Cryptographic Applications*, NIST SP 800-22 Rev 1a, April 2010.